



School of Computer Science

COMP 8347 “Internet Applications and Distributed Systems”

Summer 2026

Week 10

Cloud Deployment & DevOps (Docker, Git, CI/CD)

DR. OLENA SYROTKINA



University of Windsor

Copyright Notice

This content is protected and may not be shared, uploaded, or distributed. Any unauthorized distribution, reproduction or sharing of this material, including uploading to **CourseHero**, **Chegg**, **StuDocu** or other websites, is **strictly prohibited** and may be subject to legal action. Students are granted access to this material solely for their personal use and may not use it for any other purpose without the express written consent of the instructor. Thank you for respecting our intellectual property rights.

Agenda

- Introduction to DevOps
- Git & Version Control
- Let's practice!

Introduction to DevOps

What is DevOps?

DevOps = Development + Operations

A set of practices and tools to deliver software:

- ✓ Faster
- ✓ More reliably
- ✓ Automatically
- ✓ With fewer bugs
- ✓ With consistent environments

Traditional World vs DevOps World

Before DevOps (old way)

Developers write code → hand it to Ops → "It works on my machine" → Production breaks.

With DevOps:

Code → Build → Test → Package → Deploy → Monitor
(automated pipeline)

Introduction to DevOps

DevOps World (Modern Way)

In DevOps, we don't rely on manual steps or people remembering instructions.

We rely on an automated pipeline.

This pipeline takes the code from the developer and automatically goes through these stages:

1. Code

The developer writes code and pushes it to GitHub.

2. Build

The system automatically builds the application (for example, installs dependencies, sets up the environment, etc.)

3. Test

All tests run automatically. If something is broken, the pipeline stops and tells the developer immediately.

4. Package

If everything passes, the application is packaged. Very often, this means creating a Docker image that contains:

- the code
- the programming language
- all the libraries
- everything needed to run the app

Because of this, it runs the same on every machine.

Introduction to DevOps

DevOps World (Modern Way)

5. Deploy

The packaged application is automatically deployed to:

- a test environment, or
- production, or both.

If something goes wrong, DevOps tools notify the team immediately.

Why DevOps Helps

With DevOps we get:

✓ Faster delivery

The pipeline does everything automatically, so we can release updates many times per day if needed.

✓ Fewer bugs

Automated tests catch problems early – before users see them.

✓ More reliable deployments

Every deployment follows the same steps, not random manual steps.

✓ Same environment everywhere

A Docker image ensures the app runs the same on your laptop, the test server, and the production server.

No more “works on my machine.”

6. Monitor

Once deployed, monitoring tools check:

- Is the app running?
- Are there errors?
- Is performance okay?

Git & Version Control

What is Git?

Git is a distributed version control system. It allows:

- Tracking changes

Every time you save using Git, it remembers what changed and when.

- Rolling back

If something breaks, you can go back to any previous version safely.

- Branching

You can create separate “lines” of development for new features without touching the main code.

- Collaborating

Multiple developers can work on the same project simultaneously without overwriting each other.

- Reviewing code

Git gives a history of edits and helps teams review changes before merging them.

Distributed

- Every developer has a full copy of the repository on their machine.
- You can work offline.
- You do not depend on one central server for everything.
- If GitHub goes down, your local repo still has the complete history.
- Git was designed by Linus Torvalds (who also created Linux) to support speed, safety, and collaboration.

Git & Version Control

Daily Git Workflow – What Developers Actually Do Every Day

Most teams follow a simple structure of branches.

Here is the typical one:

Branch	Purpose
main	The production-ready code. This is what users see.
develop	The place where combined new features are tested before going to production.
feature/*	Used for each new feature. Example: feature/payment
bugfix/*	Used for fixing bugs. Example: bugfix/login-error

This structure prevents developers from breaking the production system.

Daily workflow looks like this:

1. Pick something to work on.
2. Create a branch for it (`feature/new-ui`).
3. Write code.
4. Commit frequently.
5. Push the branch to GitHub.
6. Open a Pull Request.
7. After review and tests, merge into `develop` or `main`.

This keeps the main codebase clean and stable.

Git & Version Control

Essential Git Commands:

These are the commands beginners use almost every day.

Starting a new repository

git init

This creates a new Git repository in your folder. It tells Git: “Start tracking this project.”

git remote add origin <https://github.com/yourname/project.git>

This connects your local project to GitHub. “origin” is just the nickname for that remote repository.

Everyday workflow

These three commands are the heart of Git.

1. **git add .**

This stages changes. You are telling Git: “Please prepare all these updated files to be committed.”

2. **git commit -m "meaningful message"**

This saves the version to Git history. A commit message should explain what you changed.

Examples:

- ✓ "Add login button"
- ✓ "Fix navigation bug"
- ✓ "Update README"

Git & Version Control

3. `git push`

This uploads your changes to GitHub so others can see them.

Creating a branch

```
git checkout -b feature/user-login
```

This does two things:

- ✓ Creates a new branch named **feature/user-login**.
- ✓ Moves you into that branch.

You now work safely without touching **main**.

Merging

Suppose you finished the feature. Now you want it in **main**.

1. Switch to main `git checkout main`
2. Merge the feature branch `git merge feature/user-login`

Git tries to combine the changes automatically. If there are conflicts, Git will ask you to resolve them manually.

Git & Version Control

Viewing history

```
git log --oneline --graph
```

Shows:

- ✓ A simplified history
- ✓ One commit per line
- ✓ A graph showing branches and merges

Very useful for understanding the flow of the project.

Pull Requests (PRs) – How Teams Merge Code Safely

In professional teams, developers never push directly to main.

Instead, they follow a safer process:

1. Create a branch

Example: `feature/login-page`

2. Push it to GitHub

```
git push -u origin feature/login-page
```

3. Open a Pull Request (PR)

A PR asks the team: “Please review my changes before merging them.”

Git & Version Control

4. Code is reviewed

Teammates check:

- Is the code correct?
- Are there mistakes?
- Is the style consistent?

5. CI runs tests

Continuous Integration tools automatically run:

- Unit tests
- Linting
- Security checks

If something fails, the PR cannot be merged.

6. Merge only after approval

If the review is good and tests pass:

- The feature is merged into develop or main.
- The code becomes part of the official project.

Let's Practice: Install the Tools

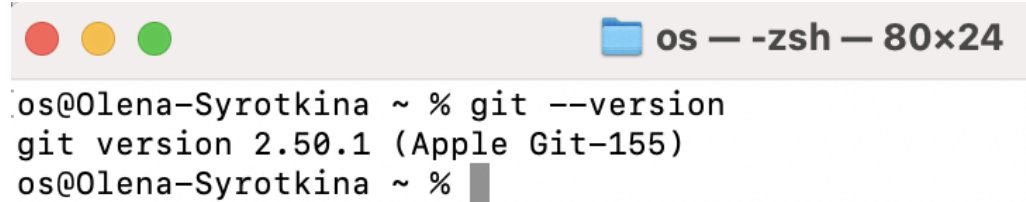
1. Install Git

Git is a tool that tracks changes in your code, lets you undo, branch, and collaborate.

- Download Git from the official site and install (default options are fine).
- After installation, in a new terminal:

<https://git-scm.com/install/>

`git --version`

A terminal window titled 'os — -zsh — 80x24' showing the command 'git --version' and its output: 'git version 2.50.1 (Apple Git-155)'. The prompt is 'os@Olena-Syrotkina ~ %'.

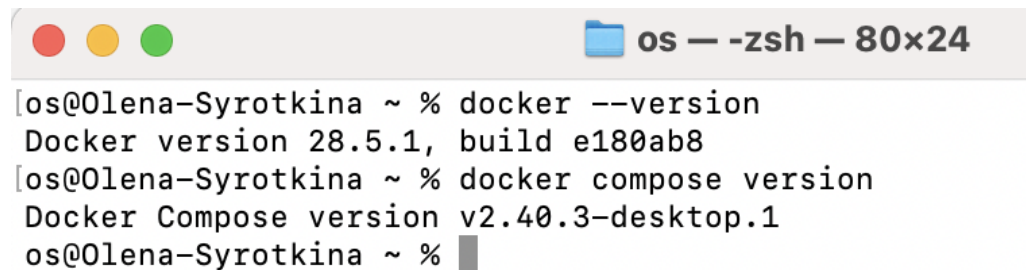
You should see something like `git version 2.x.x`.

2. Install Docker Desktop

Docker is a tool to build **containers** – tiny, self-contained environments with your code + Python + libraries.

- Download Docker Desktop for your OS and install it: <https://www.docker.com/products/docker-desktop/>
- Start **Docker Desktop**.
- In terminal, test:

`docker --version`
`docker compose version`

A terminal window titled 'os — -zsh — 80x24' showing two commands and their outputs. The first command is 'docker --version' with output 'Docker version 28.5.1, build e180ab8'. The second command is 'docker compose version' with output 'Docker Compose version v2.40.3-desktop.1'. The prompt is 'os@Olena-Syrotkina ~ %'.

Let's Practice: Install the Tools

3. Create accounts

You will need:

1. **GitHub** account → for storing code and CI/CD: <https://github.com/>
2. A free cloud provider; we will use **Render** (simple, Git-based deploy and has a free tier): <https://render.com/>

Create accounts but you don't need to do anything inside yet.

Important:

You must choose **Sign up with GitHub** *during your registration*. This is important – it allows Render to connect to your repositories automatically for CI/CD.

You will:

- Be redirected to GitHub
- GitHub will ask:
 “Do you authorize Render to access your repos?”
- Click **Authorize Render**

Let's Practice: Create a Simple Django Project

We will make a small Django app just to have something to deploy.

1.1 Make a project folder

In your terminal:

```
mkdir cloud-devops-demo  
cd cloud-devops-demo
```

1.2 Create virtual environment

Idea: Virtual env keeps your project's Python libraries separate from global ones.

Windows (PowerShell):

```
python -m venv .venv  
.venv\Scripts\activate
```

macOS / Linux:

```
python3 -m venv .venv  
source .venv/bin/activate
```

Your prompt should now show something like **(.venv)** in front.

Let's Practice: Create a Simple Django Project

We will make a small Django app just to have something to deploy.

1.3 Install Django

```
pip install django
```

1.4 Start Django project

We will call it **mysite** (you can rename but keep consistent).

```
django-admin startproject mysite .
```

You should now see files such as: `manage.py`, folder `mysite/` with `settings.py`, `urls.py`, etc.

1.5 Create a very simple app

```
python manage.py startapp hello
```

In `hello/views.py` add:

```
from django.http import HttpResponse
```

```
def home(request):  
    return HttpResponse("Hello from Django in the cloud!")
```

Let's Practice: Create a Simple Django Project

Add the **hello** app to INSTALLED_APPS

```
INSTALLED_APPS = [  
    'django.contrib.admin',  
    'django.contrib.auth',  
    'django.contrib.contenttypes',  
    'django.contrib.sessions',  
    'django.contrib.messages',  
    'django.contrib.staticfiles',  
    'hello',  
]
```

Inside **hello/urls.py**

```
from django.urls import path  
from .views import home  
  
urlpatterns = [  
    path('', home, name='hello-home'),  
]
```

Include the **hello** URLs into the main project

mysite/urls.py

```
from django.contrib import admin  
from django.urls import path, include  
  
urlpatterns = [  
    path('admin/', admin.site.urls),  
    path('hello/', include('hello.urls')),  
]
```

Apply migrations (Django internal setup):

```
python manage.py migrate
```

Run server:

```
python manage.py runserver
```

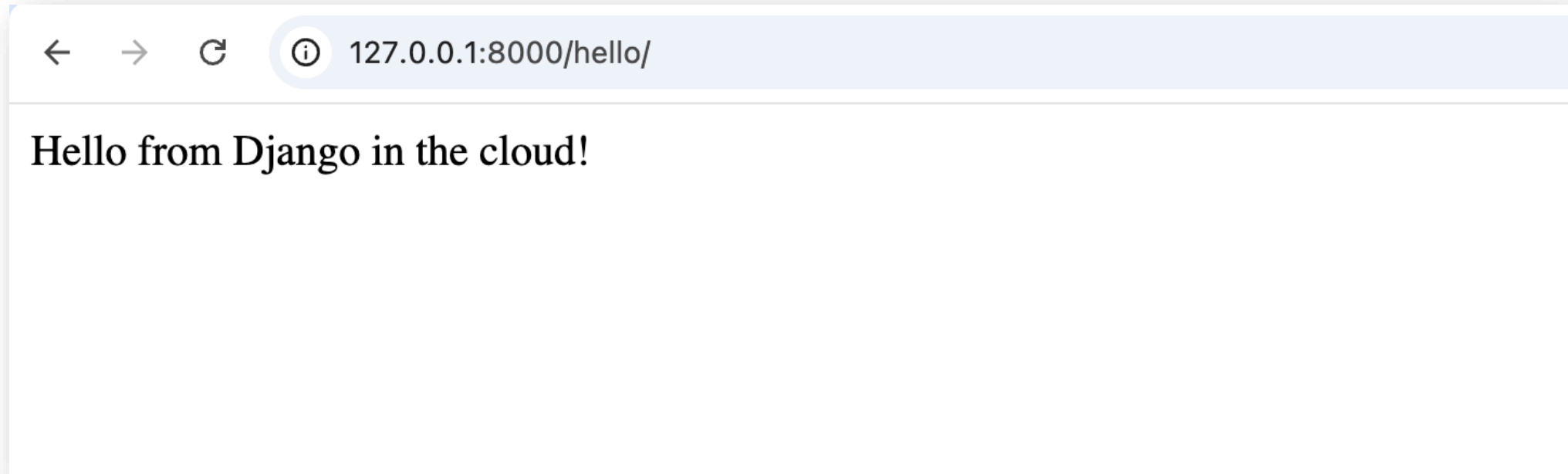
Let's Practice: Create a Simple Django Project

Open your browser: <http://127.0.0.1:8000/hello/>

You should see: **Hello from Django in the cloud!**

If yes → great! We have a working Django app.

Stop the server with **Ctrl + C** in the terminal.



Let's Practice: Put the Project Under Git Version Control

Now we use Git: track changes, make commits, and later push to GitHub.

2.1 Initialize repository

Inside **cloud-devops-demo** folder (`cd cloud-devops-demo`):

```
git init
```

This creates a hidden **.git** folder – Git's database for this project.

2.2 Create .gitignore

What is **.gitignore**?

When you use **Git**, it tracks changes to your project files.

But **some files should NEVER be tracked**, because they are:

- Automatically generated
- Temporary
- Very large
- Specific to your computer
- Sensitive

So we create a file called **.gitignore**

This file **tells Git which files and folders to ignore**.

Example:

Your virtual environment **.venv/** should NOT go to GitHub.

Your **__pycache__/** files should NOT go to GitHub.

Your **.idea/** (PyCharm settings) should NOT go to GitHub.

.gitignore = the list of files to hide from Git.

Let's Practice: Put the Project Under Git Version Control

2.2 Create .gitignore

Create `.gitignore` inside your project folder and you will see this message. Click Add.

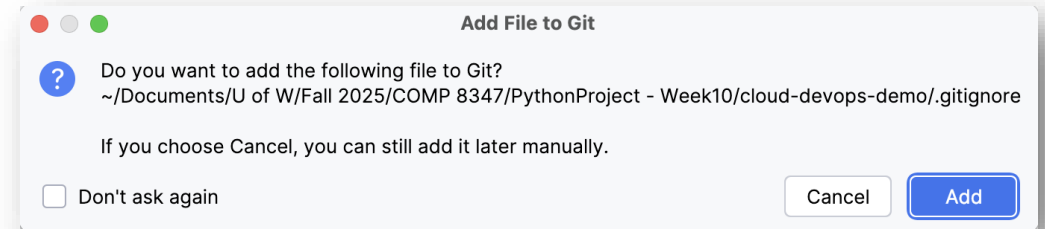
We don't want to commit `virtualenv` and temporary files. Create `.gitignore` in the project root with:

```
# Python
__pycache__/*
*.py[cod]
*.sqlite3

# Virtual env
.venv/

# Django
/staticfiles/
media/

# VSCode / PyCharm
.vscode/
.idea/
```



2.3 Check what Git sees

git status

You will see a list of “untracked files”.

```
Untracked files:
(use "git add <file>..." to include in what will be committed)
db.sqlite3
hello/
manage.py
mysite/__init__.py
mysite/asgi.py
mysite/settings.py
mysite/urls.py
mysite/wsgi.py
```

Let's Practice: Put the Project Under Git Version Control

2.4 Stage and commit

Staging = preparing files for snapshot; commit = creating the snapshot.

```
git add .  
git commit -m "Initial Django project with hello app"
```

Now Git knows about your project and we have the **first commit**.

```
(.venv) os@Olena-Syrotkina cloud-devops-demo % git commit -m "Initial Django project with hello app"  
[main (root-commit) 431c79c] Initial Django project with hello app  
Committer: Olena Syrotkina <os@Olena-Syrotkina.local>  
Your name and email address were configured automatically based  
on your username and hostname. Please check that they are accurate.  
You can suppress this message by setting them explicitly. Run the  
following command and follow the instructions in your editor to edit  
your configuration file:
```

```
git config --global --edit
```

After doing this, you may fix the identity used for this commit with:

```
git commit --amend --reset-author
```

....

Let's Practice: Containerize the Django App with Docker

We want: “My app runs the same everywhere” → Docker.

3.1 Create requirements.txt

We need a list of Python dependencies for the container.

```
pip freeze > requirements.txt
```

Later you can clean it up, but for now it's fine.

3.2 Create Dockerfile

In project root, create a file named **Dockerfile** (no extension):

```
# 1. Base image: official Python image
FROM python:3.12-slim
```

```
# 2. Avoid Python writing .pyc files and make
output unbuffered (good for logs)
```

```
ENV PYTHONDONTWRITEBYTECODE=1
ENV PYTHONUNBUFFERED=1
```

```
# 3. Set working directory inside the container
WORKDIR /app
```

```
# 4. Install system dependencies (optional,
simple case - skip extra libs)
```

```
RUN apt-get update && apt-get install -y \
    build-essential \
    && rm -rf /var/lib/apt/lists/*
```

```
# 5. Copy requirements and install Python deps
```

```
COPY requirements.txt /app/
RUN pip install --no-cache-dir -r
    requirements.txt
```

```
# 6. Copy the rest of the project into the
container
```

```
COPY . /app/
```

```
# 7. Expose port 8000 (Django dev server)
```

```
EXPOSE 8000
```

```
# 8. Command to run the app in the container
```

```
CMD ["python", "manage.py", "runserver",
    "0.0.0.0:8000"]
```

Let's Practice: Containerize the Django App with Docker

3.3 Create docker-compose.yml

Docker Compose - it's one file to define how to run containers together. For now, we just have one service: **web**. Create **docker-compose.yml** in project root:

```
version: "3.9"

services:
  web:
    build: .
    container_name: cloud_devops_demo_web
    command: python manage.py runserver 0.0.0.0:8000
    ports:
      - "8000:8000"
    volumes:
      - ./app
    env_file:
      - .env
```

We expect a **.env** file for environment variables; we will create it next.

Let's Practice: Containerize the Django App with Docker

3.4 Create .env file

✗ BUT DO NOT PUT .env into GitHub

In project root, create **.env**:

```
DJANGO_SECRET_KEY=super-secret-key-change-me  
DJANGO_DEBUG=True
```

1. DJANGO_SECRET_KEY=super-secret-key-change-me

✓ What is it?

This is your Django project's **secret key**.

Django uses it for:

- signing cookies
- cryptographic operations
- password reset tokens
- security features

Example of a real key:

```
DJANGO_SECRET_KEY=s9@3m^_#gks10c-9mqf$k5@r5u@c^z%1!$g6(7zg1q
```

✓ Why is it called “secret”?

Because if someone knows it, they can:

- ✗ forge cookies
- ✗ impersonate users
- ✗ break your app's security

So in real projects:

- This is NEVER pushed to GitHub
- It MUST be stored in .env
- You replace it with a long, random key

Example of a real key:

Let's Practice: Containerize the Django App with Docker

Now go to `mysite/settings.py` and:

At the top, import `os`:

```
import os
```

Replace `SECRET_KEY = "..."` with:

```
SECRET_KEY = os.environ.get("DJANGO_SECRET_KEY", "fallback-secret")
```

Replace `DEBUG = True` with:

```
DEBUG = os.environ.get("DJANGO_DEBUG", "False") == "True"
```

Now settings will read values from environment variables.

[3.5 Build and run with Docker Compose \(make sure that Docker is running\)](#)

In terminal (inside project folder): `docker compose build`

Let's Practice: Containerize the Django App with Docker

3.5 Build and run with Docker Compose (make sure that Docker is running)

In terminal (inside project folder): **docker compose build**

This does the following:

- ✓ Downloads base image;
- ✓ Installs Python libs;
- ✓ Copies your project into the image.

Stop it with **Ctrl + C**.

To clean containers (optional):

docker compose down

Then run: **docker compose up**

Watch the logs. When you see something like:

Starting development server at `http://0.0.0.0:8000/`

Open browser at `http://127.0.0.1:8000/hello/`.

You should again see: **Hello from Django in the cloud!**

Now your app runs **inside a container**.

Let's Practice: Commit Docker Changes and Push to GitHub

We changed many files; we want a clean history of those changes.

4.1 Stage and commit

```
git status
```

```
git add Dockerfile docker-compose.yml .env requirements.txt .gitignore mysite hello
```

```
git commit -m "Add Docker support and environment config"
```

(You can also `git add .` again if you are comfortable.)

4.2 Create GitHub repo

1. Go to GitHub → **New repository**

2. Name: **cloud-devops-demo**

3. Choose:

1. Public or private (your choice)

2. **Do NOT** add README or .gitignore there, since we already have them locally (otherwise you will need to pull & merge).

4. Create repo.

Let's Practice: Commit Docker Changes and Push to GitHub

4.2 Create GitHub repo

GitHub will show instructions such as:

Quick setup — if you've done this kind of thing before

[Set up in Desktop](#) or [HTTPS](#) [SSH](#) <https://github.com/comp8347proj/cloud-devops-demo.git>

Get started by [creating a new file](#) or [uploading an existing file](#). We recommend every repository include a [README](#), [LICENSE](#), and [.gitignore](#).

...or create a new repository on the command line

```
echo "# cloud-devops-demo" >> README.md
git init
git add README.md
git commit -m "first commit"
git branch -M main
git remote add origin https://github.com/comp8347proj/cloud-devops-demo.git
git push -u origin main
```

...or push an existing repository from the command line

```
git remote add origin https://github.com/comp8347proj/cloud-devops-demo.git
git branch -M main
git push -u origin main
```

Run these in your terminal.

If it works, your code now lives on **GitHub**.

```
(.venv) os@0lana-Syrotkina cloud-devops-demo % git push -u origin main
Username for 'https://github.com': comp8347proj
Password for 'https://comp8347proj@github.com':
remote: Invalid username or token. Password authentication is not supported for Git operations.
fatal: Authentication failed for 'https://github.com/comp8347proj/cloud-devops-demo.git/'
```

1

General

Owner *  comp8347proj / Repository name * cloud-devops-demo

✓ cloud-devops-demo is available.

Great repository names are short and memorable. How about [automatic-sniffle?](#)

Description

0 / 350 characters

2

Configuration

Choose visibility * Choose who can see and commit to this repository Public

Add README READMEs can be used as longer descriptions. [About READMEs](#) Off

Add .gitignore .gitignore tells git which files not to track. [About ignoring files](#) No .gitignore

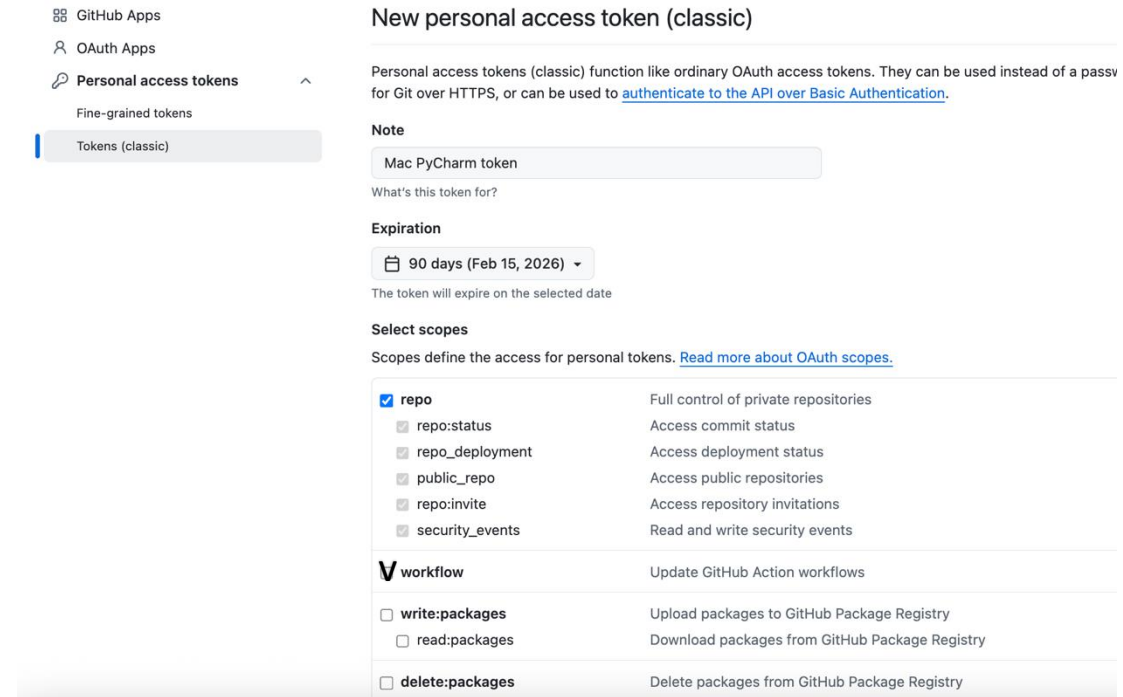
Add license Licenses explain how others can use your code. [About licenses](#) No license

Create repository

Let's Practice: Commit Docker Changes and Push to GitHub

Create a GitHub Personal Access Token (PAT)

1. Log in to GitHub as **comp8347proj** in your browser.
2. Click your profile picture (top-right) → Settings.
3. On the left, scroll all the way down to Developer settings.
4. Click Personal access tokens →
If you see "Tokens (classic)", click that.
Then click "Generate new token" →
"Generate new token (classic)".
5. Fill the form:
Note: **Mac PyCharm token** (or anything you like)
Expiration: e.g., 90 days or 180 days
Scopes: check at least:
repo (this is needed to push code)
6. Click Generate token.



The screenshot shows the GitHub interface for creating a new personal access token (classic). On the left, a sidebar lists 'GitHub Apps', 'OAuth Apps', 'Personal access tokens' (selected), 'Fine-grained tokens', and 'Tokens (classic)'. The main content area is titled 'New personal access token (classic)'. It includes a note about the function of classic tokens, a 'Note' field with 'Mac PyCharm token' entered, an 'Expiration' dropdown set to '90 days (Feb 15, 2026)', and a 'Select scopes' section. The 'repo' scope is checked, and the 'workflow' scope is also checked. Other scopes like 'repo:status', 'repo_deployment', 'public_repo', 'repo:invite', 'security_events', 'write:packages', 'read:packages', and 'delete:packages' are listed with their descriptions.

👉 IMPORTANT:

GitHub will now show you **a long string** (like ghp_1234abcd...).

This is your **token**.

- **Copy it** and keep it somewhere safe temporarily (Notes, etc.).
- You will **never see it again** after leaving the page.
- This token is your new **"password" for Git in the terminal**.

⚠️ Don't ever share this token with anyone

Let's Practice: Commit Docker Changes and Push to GitHub

Push again using the token

Back in your terminal run again: `git push -u origin main`

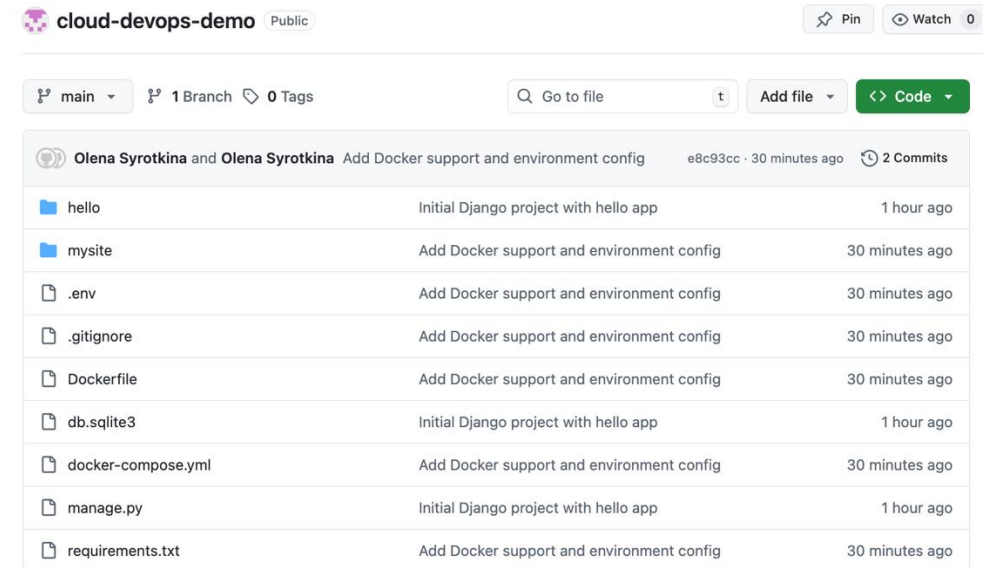
Put your username and your token.

If all is good, you should see something like:

```
(.venv) os@Olena-Syrotkina cloud-devops-demo % git push -u origin main
Username for 'https://github.com': comp8347proj
Password for 'https://comp8347proj@github.com':
Enumerating objects: 36, done.
Counting objects: 100% (36/36), done.
Delta compression using up to 10 threads
Compressing objects: 100% (34/34), done.
Writing objects: 100% (36/36), 11.62 KiB | 11.62 MiB/s, done.
Total 36 (delta 6), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (6/6), done.
To https://github.com/comp8347proj/cloud-devops-demo.git
 * [new branch]      main -> main
branch 'main' set up to track 'origin/main'.
```

```
(.venv) os@Olena-Syrotkina cloud-devops-demo %
```

That means your project is now **on GitHub**.



The screenshot shows the GitHub interface for a repository named 'cloud-devops-demo'. At the top, it indicates the repository is public and shows options to pin or watch. Below this, the main branch 'main' is selected, with 1 branch and 0 tags. A search bar and 'Add file' button are visible. The commit history table shows two commits by Olena Syrotkina. The first commit, 'Initial Django project with hello app', includes files 'hello' and 'mysite'. The second commit, 'Add Docker support and environment config', includes files '.env', '.gitignore', 'Dockerfile', 'db.sqlite3', 'docker-compose.yml', 'manage.py', and 'requirements.txt'.

Commit	Author	Message	SHA	Time	Commits
Initial Django project with hello app	Olena Syrotkina	Initial Django project with hello app	e8c93cc	30 minutes ago	2
Add Docker support and environment config	Olena Syrotkina	Add Docker support and environment config		30 minutes ago	

File	Commit	Time
hello	Initial Django project with hello app	1 hour ago
mysite	Add Docker support and environment config	30 minutes ago
.env	Add Docker support and environment config	30 minutes ago
.gitignore	Add Docker support and environment config	30 minutes ago
Dockerfile	Add Docker support and environment config	30 minutes ago
db.sqlite3	Initial Django project with hello app	1 hour ago
docker-compose.yml	Add Docker support and environment config	30 minutes ago
manage.py	Initial Django project with hello app	1 hour ago
requirements.txt	Add Docker support and environment config	30 minutes ago

Let's Practice: Add a Simple CI Pipeline (GitHub Actions)

CI = Continuous Integration → every push triggers automatic checks (tests, lint, etc.).

We will create a simple workflow that:

- ✓ Checks out the repo.
- ✓ Installs Python.
- ✓ Installs requirements.
- ✓ Runs Django tests (python manage.py test).

5.1 Create workflow file

Create folder structure: `mkdir -p .github/workflows`

Create `.github/workflows/ci.yml`:

```
name: Django CI

on:
  push:
    branches: [ "main" ]
  pull_request:
    branches: [ "main" ]
```

```
jobs:
  test:
    runs-on: ubuntu-latest

    steps:
      - name: Check out code
        uses: actions/checkout@v4

      - name: Set up Python
        uses: actions/setup-python@v5
        with:
          python-version: "3.12"

      - name: Install dependencies
        run: |
          python -m pip install --upgrade pip
          pip install -r requirements.txt

      - name: Run Django tests
        env:
          DJANGO_SECRET_KEY: test-key
          DJANGO_DEBUG: "False"
        run: |
          python manage.py migrate
          python manage.py test
```

Let's Practice: Add a Simple CI Pipeline (GitHub Actions)

5.2 Commit and push CI config

```
git add .github/workflows/ci.yml
git commit -m "Add basic Django CI workflow"
git push
```

Now go to:

- GitHub → your repo → **Actions** tab.
 - You should see workflow “Django CI” running.
 - If tests pass (Django default tests will be trivial), you’ll see ✓ green.
- You’ve now implemented **CI**.

1 workflow run

✓ Add basic Django CI workflow

main

Django CI #1: Commit [6da7653](#) pushed by [comp8347proj](#)

Let's Practice: Free Cloud Deployment (Render)

Let's deploy the Dockerized Django project to Render step by step. We will do three big things:

1. Make sure your **Dockerfile** is ready for cloud (uses PORT, gunicorn).
2. Commit & push everything to GitHub.
3. Create a Docker-based Web Service on Render.

Prepare the Docker image for deployment

Right now you have:

- cloud-devops-demo/
 - manage.py
 - Dockerfile
 - docker-compose.yml (for local dev)
 - requirements.txt
 - mysite/
 - hello/
 - .env (for local dev)

For Render we will use **only the Dockerfile** (not docker-compose).

Let's Practice: Free Cloud Deployment (Render)

1.1 Make sure gunicorn is installed

In your activated `.venv` (inside `cloud-devops-demo`):

```
pip install gunicorn
pip freeze > requirements.txt
```

This updates `requirements.txt` so your container will have **gunicorn**.

1.2 Use **PORT** env variable in Dockerfile (important for Render)

Update your `Dockerfile` to something like this:

```
# 7) Default port for local runs
ENV PORT=8000
```

```
# 8) Run Django via gunicorn, binding to $PORT
CMD gunicorn mysite.wsgi:application --bind 0.0.0.0:${PORT}
```

Why this matters:

- Render sets a `PORT` environment variable for your service (default 10000). Your app should listen on that port.
- Locally you'll still use `PORT=8000`, so it behaves like you expect.

⚠ For Render we **don't** use `docker-compose.yml` or `.env`. Render will inject environment variables in the dashboard.

Let's Practice: Free Cloud Deployment (Render)

1.3 Make ALLOWED_HOSTS safe for cloud

In `mysite/settings.py`, temporarily you can do:

```
ALLOWED_HOSTS = [ "*" ]
```

You already have:

```
SECRET_KEY = os.environ.get("DJANGO_SECRET_KEY", "fallback-secret")
DEBUG = os.environ.get("DJANGO_DEBUG", "False") == "True"
```

Perfect! Render will pass these as environment variables.

1.4 Test the Docker image locally

From `cloud-devops-demo`:

```
docker build -t cloud-devops-demo .
docker run -p 8000:8000 cloud-devops-demo
```

Open: <http://127.0.0.1:8000/hello/> → you should see your “Hello from Django...” page.

If that works, your image is ready for cloud.

Stop the container with `Ctrl + C`.

Let's Practice: Commit and Push to GitHub

From cloud-devops-demo:

```
git status
```

```
git add Dockerfile requirements.txt mysite hello .gitignore
```

```
# (and any other new files like .github/workflows/ci.yml)
```

```
git commit -m "Prepare Dockerized Django app for Render deploy"
```

```
git push
```

Now your GitHub repo contains the **Dockerfile** and code that Render will build.

1 workflow run		Event ▾	Status ▾	Branch ▾	Actor ▾
	Prepare Dockerized Django app for Render deploy Django CI #2: Commit faa91d1 pushed by comp8347proj		 main	 now  In progress	...
	Add basic Django CI workflow Django CI #1: Commit 6da7653 pushed by comp8347proj		 main	 38 minutes ago  13s	...

Let's Practice: Create a Docker-based Web Service on Render

1. Log in to Render.
2. In the dashboard click New → Web Service.
3. Choose “Build and deploy from a Git repository” and connect your GitHub.
4. Select your repo: cloud-devops-demo.

Render will detect that there is a Dockerfile and set Environment = Docker (or you may choose Docker explicitly).

Now configure:

Basic settings

- **Name:** cloud-devops-demo (anything you like)
- **Branch:** main (or whatever you use)
- **Root directory:** leave empty
- **Instance type:** choose **Free** for testing

No build/start commands needed – Dockerfile handles everything

Environment variables (very important)

Scroll to Environment Variables section and add:

- DJANGO_SECRET_KEY → some long random string
- DJANGO_DEBUG → False
- (optional but nice) PORT → 10000
 - This matches Render's default and our Dockerfile uses `${PORT}`.

The screenshot shows the 'Install on your personal account comp8347proj' window for the GitHub App. It lists permissions for repositories and user permissions. The 'Only select repositories' option is chosen, and 'comp8347proj/cloud-devops-demo' is selected. The 'Install' button is highlighted in green.

Install on your personal account comp8347proj

for these repositories:

☐ All repositories
This applies to all current and future repositories owned by the resource owner. Also includes public repositories (read-only).

☒ Only select repositories
Select at least one repository. Also includes public repositories (read-only).

Select repositories ▼

Selected 1 repository.

comp8347proj/cloud-devops-demo ×

with these permissions:

- ✓ Read access to Dependabot alerts, administration, code, and metadata
- ✓ Read and write access to actions, checks, commit statuses, deployments, environments, issues, pull requests, repository hooks, and workflows

User permissions
Render can also request users' permission to the following resources. These permissions will be requested and authorized on an individual-user basis.

- ✓ Read access to email addresses

Install Cancel

Next: you'll be directed to the GitHub App's site to complete setup.

Let's Practice: Create a Docker-based Web Service on Render

Click **Create Web Service**
/ **Deploy**.

Render will now:

- Clone your repo
 - Build the Docker image using your Dockerfile
 - Run the container
 - Check that something is listening on 0.0.0.0:\$PORT
- You can watch the logs live in Render's dashboard.

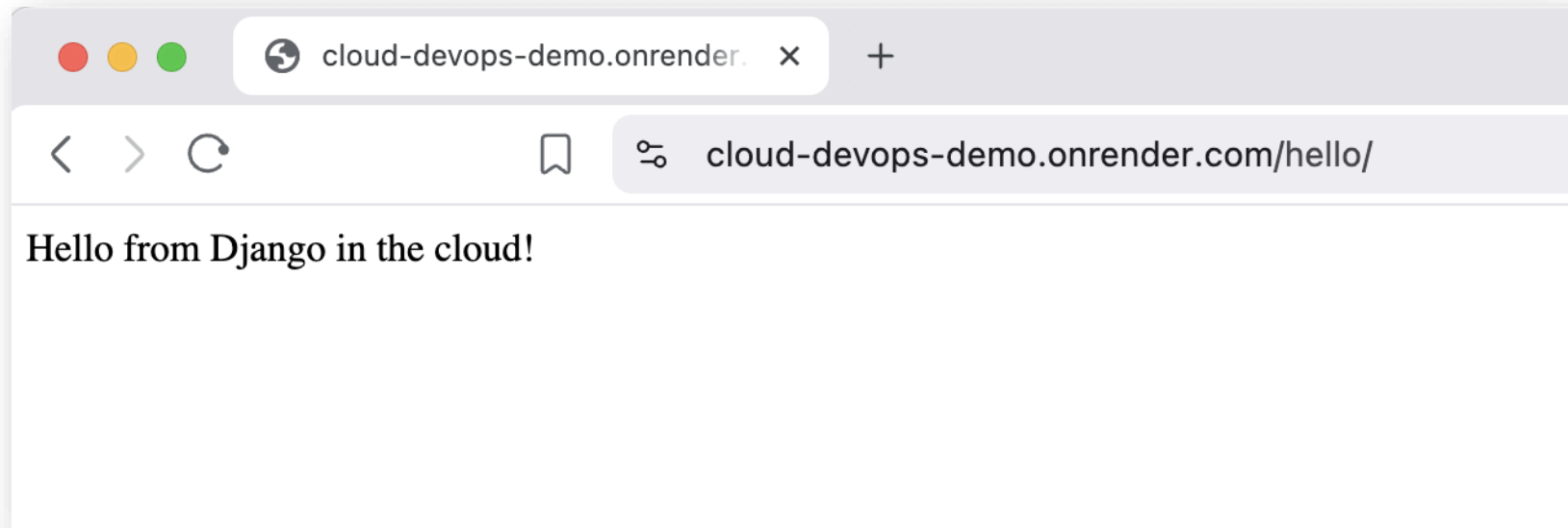
The screenshot displays the Render dashboard for a project named 'cloud-devops-demo'. The top navigation bar includes 'My project', 'Production', and 'cloud-devops-demo'. The main section shows the service ID 'srv-d4e0skur433s738cpc60' and a 'Free' tier with an 'Upgrade your instance' link. Below this, the service ID is repeated, and a link to the deployment is provided: 'https://cloud-devops-demo.onrender.com'. A warning banner indicates that the free instance will spin down with inactivity, with a link to 'Upgrade now'. The deployment status is 'Live' as of November 18, 2025, at 1:12 AM. The logs section shows the following sequence of events:

- Nov 18 01:13:18 AM: #13 Writing cache image manifest sha256:021b88v3b0c2f1094ca20d31e240634c0/902/83a041e0e8e344/260/4040401 done
- Nov 18 01:13:18 AM: #13 DONE 1.1s
- Nov 18 01:13:20 AM: Pushing image to registry...
- Nov 18 01:13:26 AM: Upload succeeded
- Nov 18 01:13:31 AM: ==> Deploying...
- Nov 18 01:13:43 AM: [2025-11-18 06:13:43 +0000] [7] [INFO] Starting gunicorn 23.0.0
- Nov 18 01:13:43 AM: [2025-11-18 06:13:43 +0000] [7] [INFO] Listening at: http://0.0.0.0:10000 (7)
- Nov 18 01:13:43 AM: [2025-11-18 06:13:43 +0000] [7] [INFO] Using worker: sync
- Nov 18 01:13:43 AM: [2025-11-18 06:13:43 +0000] [26] [INFO] Booting worker with pid: 26
- Nov 18 01:13:51 AM: ==> Your service is live 🎉
- Nov 18 01:13:51 AM: ==>
- Nov 18 01:13:51 AM: ==> Available at your primary URL https://cloud-devops-demo.onrender.com
- Nov 18 01:13:51 AM: ==>
- Nov 18 01:13:51 AM: ==>

Let's Practice: Create a Docker-based Web Service on Render

Test in the browser:

<https://cloud-devops-demo.onrender.com/hello/>



You should see your Django “Hello” page, now running in a **Docker container in the cloud!**